
Universidad Aeronáutica de Querétaro

Embedded Systems Laboratory

SCADE One Laboratory Series

Practice #2

ESP32 Development Environment Setup

Elaborated by: Leonardo Franco García

Student ID: 12148

Date: July 10, 2026

Full name: _____

Student ID: _____

Date: _____

1 Learning Objectives

Learning Objectives

By the end of this practice, the student will be able to:

- ✓ **Install and verify ESP-IDF v5.x:** set up the complete Espressif toolchain (Python environment, CMake, Ninja, and the Xtensa GCC cross-compiler) and confirm it with `idf.py -version`.
- ✓ **Explain the ESP-IDF project anatomy:** identify the role of the top-level and component-level `CMakeLists.txt`, the `main` component, and the `components/` folder where SCADE-generated code will live from Practice 3 onwards.
- ✓ **Execute the full build-flash-run chain:** compile a firmware image, flash it over USB, and observe a 1 Hz blink on the on-board LED, proving the toolchain works end to end.
- ✓ **Operate the UART diagnostic channel:** use `idf.py monitor` and the `ESP_LOG` macros to observe the internal state of the firmware, the observability mechanism used in every subsequent practice.

2 Introduction

In avionics programs, target integration failures traced back to a misconfigured build environment are among the most expensive to diagnose, because they masquerade as model or code defects. Before any SCADE-generated code reaches the ESP32, the toolchain that compiles, links, and flashes it must be installed and proven trustworthy in isolation. This practice dedicates one full session to exactly that: the landing gear control system will eventually run as C code on this board, and every practice from Practice 3 onward assumes the chain `edit` → `build` → `flash` → `monitor` works without surprises. No SCADE modeling is performed here; the deliverable is a verified environment and a first firmware whose behavior is observable both physically (LED) and through the UART diagnostic channel.

3 Bill of Materials

#	Component / Software	Qty	Notes
1	SCADE One (Ansys)	1	Not used in this practice (see note)
2	ESP32 DevKit V1	1	Any ESP32 variant with on-board LED (GPIO 2)
3	USB-A to Micro-USB cable	1	Must be a <i>data</i> cable, not charge-only
4	ESP-IDF (v5.x)	1	Installed and verified in this practice
5	Host PC	1	Windows 10/11; ≥ 8 GB free disk space

Note

This practice is hardware-plus-toolchain only: SCADE One is not opened at any point. It remains in the Bill of Materials for course continuity — the project structure created today is the exact structure that will receive KCG-generated code in Practice 3.

4 Theory & Block Reference

No SCADE One blocks are introduced in this practice. The “blocks” of this session are the building elements of the ESP-IDF toolchain and the minimal C API surface required to blink an LED and log over UART. Each is documented in the same reference format so it can be consulted from later practices.

4.1. The ESP-IDF build pipeline

ESP-IDF is not a single compiler but a pipeline of cooperating tools. `idf.py` is a Python front-end that orchestrates CMake (project configuration), Ninja (parallel build execution), the Xtensa GCC cross-compiler (translation of C to Xtensa LX6 machine code), and `esptool.py` (serial flashing). Understanding which layer failed is the key diagnostic skill of this practice: a CMake error means the project description is wrong; a GCC error means the C code is wrong; an esptool error means the serial link or boot mode is wrong.

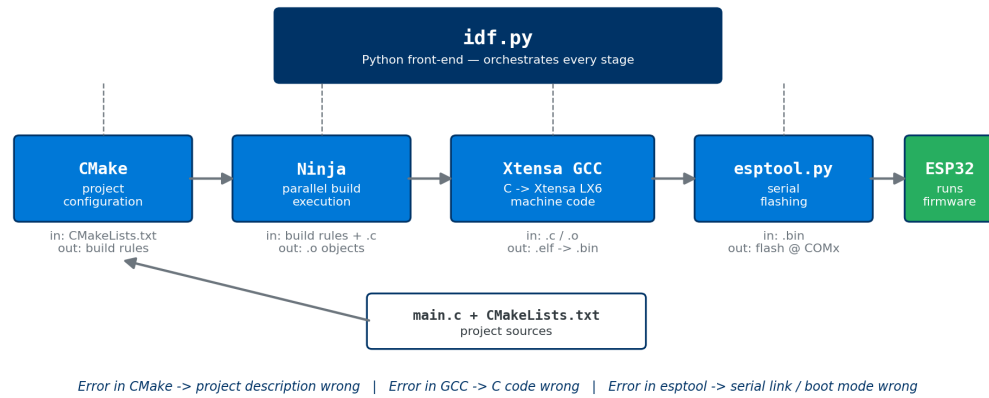


Figure 1: The ESP-IDF build pipeline: `idf.py` orchestrating CMake, Ninja, Xtensa GCC, and `esptool.py`. Annotate each stage with its input and output artifact.

Block: idf.py

```

Commands
add-dependency    Add dependency to the manifest file.
all               Aliases: build, build the project.
app              Build only the app.
app-flash        Flash the app only.
bootloader       Build only bootloader.
bootloader-flash Flash bootloader only.
build-system-targets
                  Print list of build system targets.
clang-check       Run clang-tidy check under current folder, write the output into "warnings.txt".
clang-hl-report   generate html report to "html_report" folder by reading "warnings.txt" (may take a
                  few minutes). This feature requires extra dependency "codereport". Please install
                  this by running "pip install codereport".
clean             Delete build output files from the build directory.
console          Run JTAG configuration server.
confserver        Create core dump ELF file and run GDB debug session with this file.
coredump-debug   Print crashed task's registers, callstack, list of available tasks in the system,
coredump-info    memory regions and contents of memory stored in core dump (VRs and stacks).
create-component Create a new component.
create-manifest  Create manifest for specified component.
create-project   Create a new project.
create-project-from-example Create a project from an example in the ESP Component Registry.
diag            Create diagnostic report.
docs            Open web browser with documentation for ESP-IDF.
efuse-burn       Burn the efuse with the specified name.
efuse-burn-key   Burn a 256-bit key to EFUSE: BLOCK2, flash_encryption, BLOCK2, secure_boot_v1,
efuse-common-table Generate C-source for IDF's efuse fields.
efuse-custom-table Generate C-source for user's efuse fields.
efuse-dump       Dump the hex values of all efuses.
efuse-read-protect Disable writing to the efuse with the specified name.
efuse-memory     Get the memory of the efuses.
efuse-write-protect Disable writing to the efuse with the specified name.
encrypted-flash  Flash the encrypted app only.
erase-flash      Erase entire flash chip.
erase-otadata    Erase otadata partition.
flash            Flash the project.
fullclean        Delete the entire build directory contents.
gdb              Run the GDB.
idfui            GDB UI in default browser.
idfui           GDB UI mode.

```

Figure 2: Subcommand list printed by `idf.py -help`, showing the command-line surface of the tool. This practice uses only `set-target`, `build`, `flash`, and `monitor`; the alias `all` is equivalent to `build`.

Subcommands (the “ports” of the tool):

- **set-target esp32:** selects the chip and generates the initial CMake cache.
- **menuconfig:** opens the Kconfig text-mode configuration editor.
- **build:** compiles and links; produces `build/*.bin`.
- **flash -p <PORT>:** writes bootloader, partition table, and application to the board.
- **monitor -p <PORT>:** opens the UART console at 115200 baud; exit with `Ctrl+]`.

Behavior:

Subcommands compose: `idf.py flash` `monitor` builds if needed, flashes, then attaches the monitor in one invocation. `idf.py` only works inside a terminal where the ESP-IDF environment has been exported; in a plain terminal every subcommand fails with “command not found.”

Block: CMakeLists.txt (project structure)

```
C:\Espressif\frameworks\esp-idf-v5.5.4>cd C:\esp
C:\esp>mkdir lg_env_check lg_env_check\main lg_env_check\components
C:\esp>cd lg_env_check
C:\esp\lg_env_check>tree /f C:\esp\lg_env_check
Listado de rutas de carpetas para el volumen OS
El número de serie del volumen es 000001FC D270:E55C
C:\ESP\LG_ENV_CHECK
├── components
└── main
C:\esp\lg_env_check>
```

Figure 3: Directory tree of a minimal ESP-IDF project — annotated view.

Elements:

- **Top-level CMakeLists.txt:** names the project and includes the ESP-IDF build system.
- **main/:** the mandatory entry component; contains `app_main()`.
- **main/CMakeLists.txt:** registers the component's source files via `idf_component_register`.
- **components/:** optional folder; each subfolder with its own `CMakeLists.txt` becomes a self-contained library.
- **sdkconfig:** generated configuration; never edited by hand.

Behavior:

Every folder inside `components/` is discovered and built automatically. This is the integration mechanism for the entire course: in Practice 3, KCG output is dropped into `components/scade_generated/` and the build system links it without any modification to `main`. A source file not listed in `idf_component_register` is silently ignored — the most common cause of “undefined reference” link errors.

Block: GPIO driver (gpio_*)

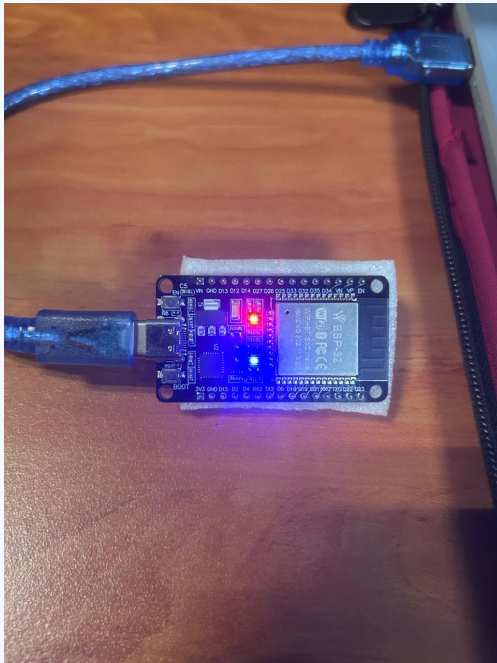


Figure 4: GPIO output configuration and the on-board LED on GPIO 2 — annotated view.

API surface used in this practice:

- **gpio_reset_pin(pin):** returns the pin to its default state, detaching any previous peripheral.
- **gpio_set_direction(pin, GPIO_MODE_OUTPUT):** configures the pin as a push-pull output.
- **gpio_set_level(pin, level):** drives the pin to 0 or 1.

Behavior:

On the DevKit V1 the on-board LED is wired to GPIO 2 and is active-high. Writing to a pin that has not been configured as an output compiles cleanly but has no physical effect — a silent failure mode worth remembering. GPIO 6–11 are reserved for the internal flash and must never be used.

Block: ESP_LOG (UART diagnostics)

Figure 5: ESP_LOGI output as rendered by `idf.py monitor` — annotated view.

Macro family:

- **ESP_LOGE(TAG, fmt, ...):** error (red).
- **ESP_LOGW(TAG, fmt, ...):** warning (yellow).
- **ESP_LOGI(TAG, fmt, ...):** info (green).
- **TAG:** a static const char* identifying the module; used to filter output per component.

Behavior:

Each message is timestamped in milliseconds since boot and routed to UART0 at 115 200 baud. Unlike raw `printf`, log verbosity is adjustable per tag at runtime and globally through `menuconfig`. This channel is the sole window into the internal state of the landing gear control system until Practice 7 adds physical actuators — treat its message format as an interface, not decoration.

5 Worked Example

5.1. Objective

Install ESP-IDF, create a project from the official `hello_world` template, and run it on the board. This exercises the full pipeline — configure, build, flash, monitor — without writing any C code, isolating environment problems from programming problems.

5.2. Step-by-Step

1. Download and run the ESP-IDF Windows installer.

From <https://dl.espressif.com/dl/esp-idf/>, download the *Offline Installer* for the latest v5.x release. Accept the defaults: it installs Python, Git, CMake, Ninja, the Xtensa toolchain, and the USB-serial drivers (CP210x / CH340) in one pass.

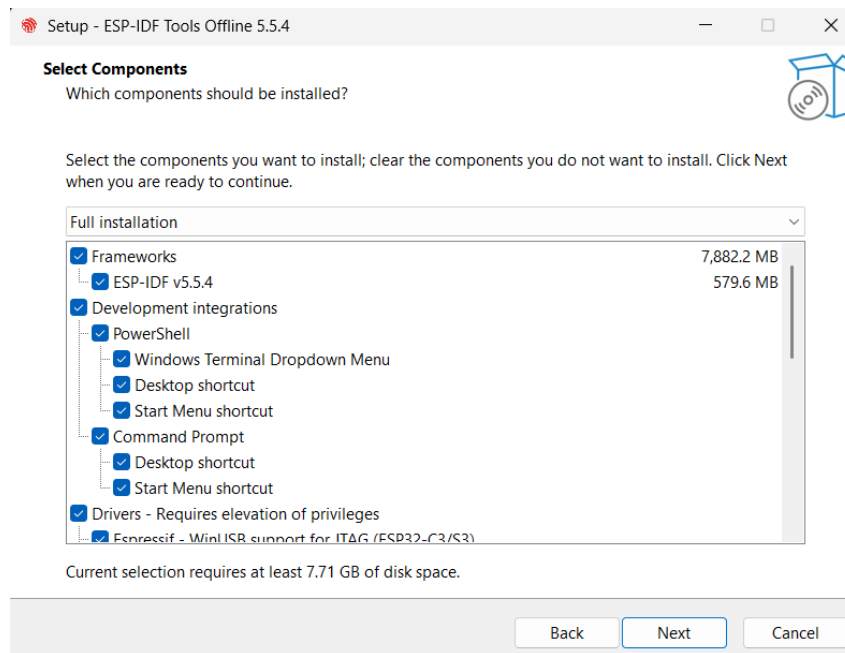


Figure 6: ESP-IDF offline installer — component selection screen.

2. Open the “ESP-IDF 5.x CMD” terminal and verify the installation.

Use the Start-menu shortcut created by the installer (it exports the environment automatically). Run:

```
idf.py --version
```

A version string such as **ESP-IDF v5.5.4** confirms the toolchain is on the PATH. The first invocation can take up to a minute while the Python environment warms up; subsequent runs respond immediately.

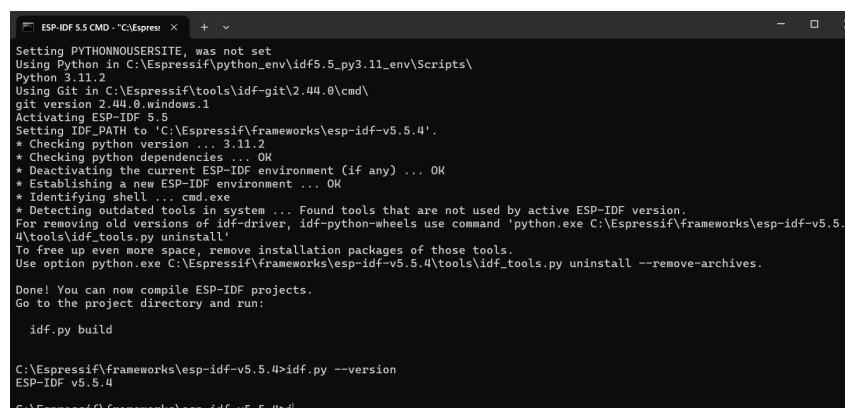


Figure 7: idf.py -version output in the ESP-IDF terminal.

3. Connect the ESP32 and verify the USB-serial driver.

Plug the board in with the USB data cable, open Windows Device Manager, and confirm the device appears under *Ports (COM & LPT)* as *Silicon Labs CP210x USB to UART Bridge (COMx)* or *USB-SERIAL CH340 (COMx)*; note the number, e.g. **COM8**. The driver is normally already present: the ESP-IDF installer

bundles it and Windows 10/11 also installs the CP210x automatically through Windows Update. If the board instead shows under *Other devices* with a warning icon, install the driver for your bridge chip (the small IC next to the USB connector): CP210x from https://www.silabs.com/documents/public/software/CP210x_Universal_Windows_Driver.zip (extract, right-click `silabser.inf` → *Install*) or CH340 from https://www.wch-ic.com/downloads/CH341SER_EXE.html. If nothing appears at all, the cable is charge-only; replace it with a data cable before any build.



Figure 8: Device Manager showing the ESP32 on COM8.

4. **Create a projects folder, copy the `hello_world` example, and set the target.** The ESP-IDF terminal always opens inside the framework directory; never create projects there. Move to a dedicated projects folder first, then copy the example (type the paths exactly as shown, with no spaces around the backslashes):

```
1 mkdir C:\esp
2 cd C:\esp
3 xcopy /e /i %IDF_PATH%\examples\get-started\hello_world hello_world
4 cd hello_world
5 idf.py set-target esp32
```

`set-target` runs the full CMake configuration and ends with Build files have been written to: `C:/esp/hello_world/build`.

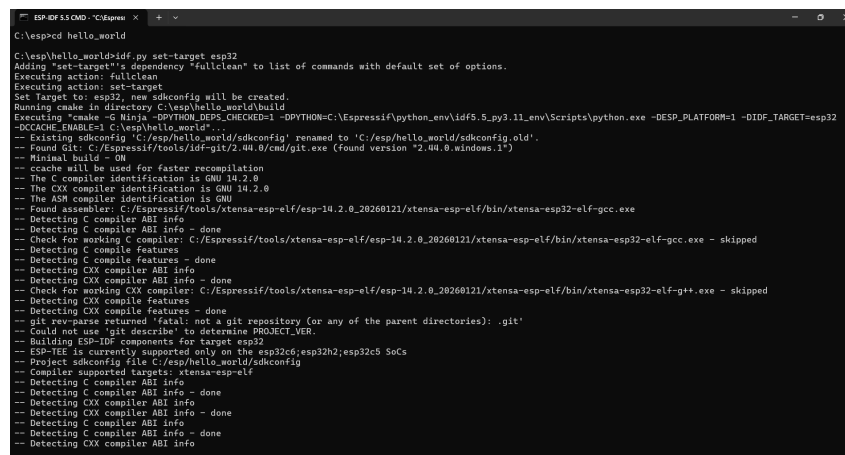


Figure 9: `idf.py set-target esp32` completing the CMake configuration.

5. **Build, flash, and monitor in one command.**

Replace COM8 with your port:

```
1 idf.py -p COM8 flash monitor
```

The first build compiles the entire IDF and takes several minutes; subsequent builds are incremental.

```

C:\esp\hello_world>idf . py -p COM8 flash monitor
"idf" no se reconoce como un comando interno o externo,
programa o archivo por lotes ejecutable.

C:\esp\hello_world>idf.py -p COM8 flash monitor
Executing action: flash
Running ninja in directory C:\esp\hello_world\build
Executing "ninja flash"...
[35/489] Generating ../../partition_table/partition-table.bin
Partition table binary generated. Contents:
*****
# ESP-IDF Partition Table
# Name, Type, SubType, Offset, Size, Flags
nvs,data,nvs,0x9000,24K,
phy_init,data,phy,0xf000,4K,
factory,app,factory,0x10000,1M,
*****
[435/489] No download step for 'bootloader'|

```

Figure 10: Successful flash followed by the monitor attaching.

6. Verify the output and detach the monitor.

The console must print `Hello world!` followed by chip information, then restart every 10 seconds. Exit the monitor with `Ctrl+]`. The environment is now proven end to end.

```

[205] main-task: calling app.main()
Hello world!
This is esp32 chip with 2 CPU core(s), WiFi/BT/BLE, silicon revision v3.1, 2MB external flash
Minimum free heap size: 394988 bytes
Restarting in 10 seconds...
Restarting in 9 seconds...
Restarting in 8 seconds...
Restarting in 7 seconds...
Restarting in 6 seconds...
Restarting in 5 seconds...
Restarting in 4 seconds...
Restarting in 3 seconds...
Restarting in 2 seconds...
Restarting in 1 seconds...
Restarting now.
ets Jul 29 2019 12:21:46
rst:0xc (SW_CPU_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode DIO, clock div:2
load 0x3fff9039, len 6288
load 0x40078000, len 15864
load 0x40080000, len 3928
-- 0x40080000 - invalid pc placeholder at C:/Espressif/frameworks/esp-idf-v5.5.4/components/stens/stens_vectors.5:2235
entry 0x40080000
[27] boot: ESP-IDF v5.5.4 2nd stage bootloader
[27] boot: compile time Jul 3 2026 11:01:39
[27] boot: Multicore bootloader
[27] boot: chip revision: v3.1
[31] boot: esp32: SPI Speed : 80MHz
[31] boot: esp32: SPI Mode : DIO
[39] boot: esp32: SPI Flash Size : 2MB
[42] boot: Reading new early entropy source...
[47] boot: Partition Table:
[47] boot:
[47] boot: # Label Usage Type ST Offset Length
[47] boot: # Name
[47] boot: 0 nvs WiFi data 01 02 00000000 00000000
[47] boot: 1 phy_init RF data 01 01 00001000 00001000
[47] boot: 2 factory factory app 00 00 00010000 00100000

```

Figure 11: `hello_world` output cycling in the monitor.

6 Main Activity

6.1. Description

Create a new ESP-IDF project named `lg_env_check` from scratch — not from an example — reproducing the minimal project structure by hand so that every file's purpose is understood. The firmware must blink the on-board LED at exactly 1 Hz and report each LED transition over UART using `ESP_LOGI` with the tag `LG_ENV`. This project becomes the container that receives SCADE-generated code in Practice 3, so its structure — not just its behavior — is part of the deliverable.

6.2. Functional Specifications

- **Spec 1:** The project is created manually with the exact structure: `top-level CMakeLists.txt`, `main/` with its own `CMakeLists.txt` and `main.c`, and an empty `components/` folder committed as part of the tree.

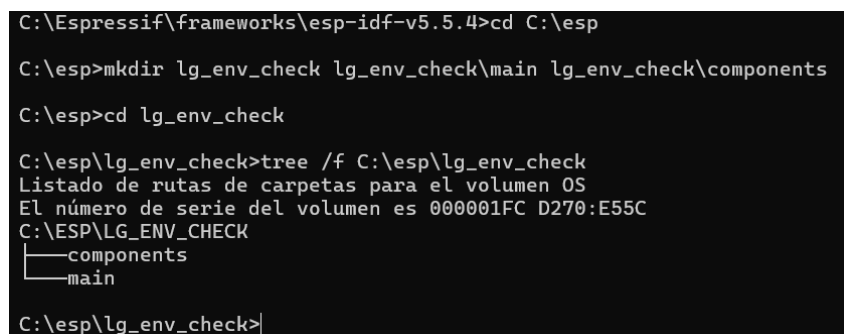
- **Spec 2:** The on-board LED (GPIO 2) blinks at $1\text{ Hz} \pm 5\%$ — 500 ms ON, 500 ms OFF — using `vTaskDelay(pdMS_TO_TICKS(500))` for timing.
- **Spec 3:** Every transition is logged with tag `LG_ENV` in the format `LED -> ON / LED -> OFF`, plus a cycle counter that increments once per full period.
- **Spec 4:** A single boot banner reports the project name and ESP-IDF version (`IDF_VER`) exactly once at startup.
- **Spec 5:** The firmware builds with zero warnings and flashes with a single `idf.py -p <PORT> flash monitor` invocation.

6.3. Activity Steps

1. Create the project skeleton by hand.

In the ESP-IDF terminal, move to your projects folder first — never create projects inside `C:\Espressif` — and type the paths with no spaces around the backslashes:

```
1 cd C:\esp
2 mkdir lg_env_check lg_env_check\main lg_env_check\components
3 cd lg_env_check
```



```
C:\Espressif\frameworks\esp-idf-v5.5.4>cd C:\esp
C:\esp>mkdir lg_env_check lg_env_check\main lg_env_check\components
C:\esp>cd lg_env_check
C:\esp\lg_env_check>tree /f C:\esp\lg_env_check
Listado de rutas de carpetas para el volumen OS
El número de serie del volumen es 000001FC D270:E55C
C:\ESP\LG_ENV_CHECK
├── components
└── main
C:\esp\lg_env_check>
```

Figure 12: Manually created project tree for `lg_env_check`.

2. Write the top-level `CMakeLists.txt`.

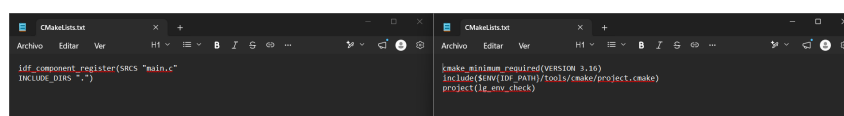
Create `lg_env_check/CMakeLists.txt` with:

```
1 cmake_minimum_required(VERSION 3.16)
2 include($ENV{IDF_PATH}/tools/cmake/project.cmake)
3 project(lg_env_check)
```

3. Register the main component.

Create `main/CMakeLists.txt` with:

```
1 idf_component_register(SRCS "main.c"
2                       INCLUDE_DIRS ".")
```



The image shows two side-by-side windows of a code editor. The left window shows the `main/CMakeLists.txt` file with the following content: `idf_component_register(SRCS "main.c" INCLUDE_DIRS ".")`. The right window shows the `lg_env_check/CMakeLists.txt` file with the following content: `cmake_minimum_required(VERSION 3.16)`, `include($ENV{IDF_PATH}/tools/cmake/project.cmake)`, and `project(lg_env_check)`.

Figure 13: Both `CMakeLists.txt` files in the editor.

4. Write main/main.c.

Implement `app_main()` so that it prints the boot banner, configures GPIO 2 as an output, and enters an infinite loop that toggles the LED every 500 ms, logging each transition and the cycle count per Spec 3. Use only the APIs documented in Section 4.

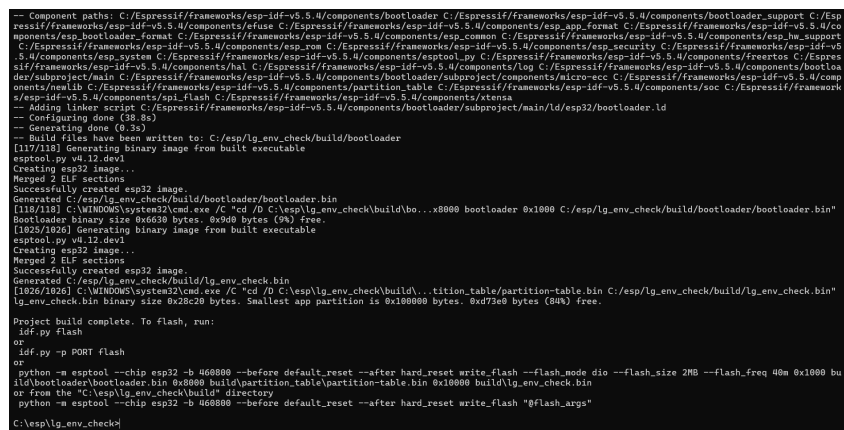
```

1 #include "freertos/FreeRTOS.h"
2 #include "freertos/task.h"
3 #include "driver/gpio.h"
4 #include "esp_log.h"
5 #define LED_PIN GPIO_NUM_2
6 static const char *TAG = "LG_ENV";
7 void app_main(void)
8 {
9     ESP_LOGI(TAG, "lg_env_check | ESP-IDF %s", IDF_VER);
10    gpio_reset_pin(LED_PIN);
11    gpio_set_direction(LED_PIN, GPIO_MODE_OUTPUT);
12    uint32_t cycle = 0;
13    while (1) {
14        gpio_set_level(LED_PIN, 1);
15        ESP_LOGI(TAG, "LED -> ON | cycle=%lu", (unsigned long)cycle);
16        vTaskDelay(pdMS_TO_TICKS(500));
17        gpio_set_level(LED_PIN, 0);
18        ESP_LOGI(TAG, "LED -> OFF | cycle=%lu", (unsigned long)cycle);
19        vTaskDelay(pdMS_TO_TICKS(500));
20        cycle++;
21    }
22 }

```

5. Set the target and build.

Run `idf.py set-target esp32` followed by `idf.py build`. Resolve any warning before continuing — Spec 5 requires a clean build.



```

-- Component paths: C:/Espressif/frameworks/esp-idf-v5.5.4/components/bootloader C:/Espressif/frameworks/esp-idf-v5.5.4/components/bootloader-support C:/Espressif/frameworks/esp-idf-v5.5.4/components/efuse C:/Espressif/frameworks/esp-idf-v5.5.4/components/esp_app_format C:/Espressif/frameworks/esp-idf-v5.5.4/components/esp_bootloader_format C:/Espressif/frameworks/esp-idf-v5.5.4/components/esp_common C:/Espressif/frameworks/esp-idf-v5.5.4/components/esp_hw_support C:/Espressif/frameworks/esp-idf-v5.5.4/components/esp_rom C:/Espressif/frameworks/esp-idf-v5.5.4/components/esp_security C:/Espressif/frameworks/esp-idf-v5.5.4/components/esp_system C:/Espressif/frameworks/esp-idf-v5.5.4/components/esptool.py C:/Espressif/frameworks/esp-idf-v5.5.4/components/freertos C:/Espressif/frameworks/esp-idf-v5.5.4/components/hal C:/Espressif/frameworks/esp-idf-v5.5.4/components/log C:/Espressif/frameworks/esp-idf-v5.5.4/components/bootloader/subproject/main C:/Espressif/frameworks/esp-idf-v5.5.4/components/bootloader/subproject/components/nlcc C:/Espressif/frameworks/esp-idf-v5.5.4/components/newlib C:/Espressif/frameworks/esp-idf-v5.5.4/components/partition_table C:/Espressif/frameworks/esp-idf-v5.5.4/components/soc C:/Espressif/frameworks/esp-idf-v5.5.4/components/spi_flash C:/Espressif/frameworks/esp-idf-v5.5.4/components/xtensa
-- Adding linker script C:/Espressif/frameworks/esp-idf-v5.5.4/components/bootloader/subproject/main/ld/esp32/bootloader.ld
-- Configuring done (0.8s)
-- Generating done (0.2s)
-- Build files have been written to: C:/esp/lg_env_check/build/bootloader
[117/118] Generating binary image from built executable
esptool.py v4.12.dev1
Creating esp32 image...
Merged 2 ELF sections
Successfully created esp32 image.
Generated C:/esp/lg_env_check/build/bootloader/bootloader.bin
[118/118] C:\WINDOWS\system32\cmd.exe /C "cd /D C:/esp/lg_env_check/build/bo...x0800 bootloader 0x1000 C:/esp/lg_env_check/build/bootloader/bootloader.bin"
Bootloader binary size 0x6638 bytes, 0x900 bytes (9%) free.
[1025/1026] Generating binary image from built executable
esptool.py v4.12.dev1
Creating esp32 image...
Merged 2 ELF sections
Successfully created esp32 image.
Generated C:/esp/lg_env_check/build/lg_env_check.bin
[1026/1026] C:\WINDOWS\system32\cmd.exe /C "cd /D C:/esp/lg_env_check/build/...tion table/partition-table.bin C:/esp/lg_env_check/build/lg_env_check.bin"
lg_env_check.bin binary size 0x2629 bytes. Smallest app partition is 0x10000 bytes, 0x072e0 bytes (8%) free.

Project build complete. To flash, run:
idf.py flash
or
python -m esptool --chip esp32 -b 4608000 --before default_reset --after hard_reset write_flash --flash_mode dio --flash_size 2MB --flash_freq 40m 0x1000 boot
loader.bin 0x8000 build/partition-table.bin 0x10000 build/lg_env_check.bin
or from the "C:/esp/lg_env_check/build" directory
python -m esptool --chip esp32 -b 4608000 --before default_reset --after hard_reset write_flash @flash_args"
C:/esp/lg_env_check\

```

Figure 14: Clean build summary showing the binary size.

6. Flash the board and attach the monitor.

Run `idf.py -p COM8 flash` monitor (your port may differ).

7. Verify timing against the specification.

Confirm with the monitor timestamps that consecutive LED -> ON messages are $1000\text{ ms} \pm 50\text{ ms}$ apart, and visually confirm the LED blink matches the log.

```

(133) esp_image: segment 0: paddr=0002f0b8 vaddr=00000000 size=00e1b8 ( 36376) load
(187) esp_image: segment 5: paddr=0002f0b8 vaddr=00000000 size=000200 (   32) load
(194) boot: Loading app from partition at offset 0x100000
(194) boot: Disabling RNG early entropy source...
(195) cpu_start: MultiCore app
(196) cpu_start: GPIO 3 and 1 are used as console UART I/O pins
(196) cpu_start: Pre cpu start user code
(197) cpu_start: cpu freq: 160000000 Hz
(196) app_init: Application Information:
(198) app_init: Project name: lg_env_check
(199) app_init: App version: 1
(197) app_init: Compile time: Jul 9 2026 13:18:16
(192) app_init: ELF file SHA256: 6636097c...
(199) app_init: ESP-IDF: v5.5.4
(203) efuse_init: Min chip rev: v0.8
(205) efuse_init: Max chip rev: v3.99
(205) efuse_init: Chip rev: v3.1
(212) heap_init: Initializing. RAM available for dynamic allocation:
(219) heap_init: At 3FFA2000 len 00012000 (5 KiB): DRAM
(224) heap_init: At 3FFB1000 len 0002CF08 (179 KiB): DRAM
(229) heap_init: At 3FFC0000 len 00002A08 (14 KiB): D/IRAM
(234) heap_init: At 3FFC3000 len 00012C00 (111 KiB): D/IRAM
(240) heap_init: At 40000000 len 00011000 (76 KiB): IRAM
(240) spi_flash: detected chip: generic
(240) spi_flash: Flash io: dio
(252) spi_flash: Detected size(4096K) larger than the size in the binary image header(2048K). Using the size in the binary image header.
(264) main_task: Started on CPU0
(270) main_task: Calling app_main()
(270) LG_ENV: lg_env_check | ESP-IDF v5.5.4
(270) LG_ENV: LED -> ON | cycle=0
(770) LG_ENV: LED -> OFF | cycle=0
(1270) LG_ENV: LED -> ON | cycle=1
(1770) LG_ENV: LED -> OFF | cycle=1
(2270) LG_ENV: LED -> ON | cycle=2
(2770) LG_ENV: LED -> OFF | cycle=2
(3270) LG_ENV: LED -> ON | cycle=3
(3770) LG_ENV: LED -> OFF | cycle=3
(4270) LG_ENV: LED -> ON | cycle=4
(4770) LG_ENV: LED -> OFF | cycle=4
(5270) LG_ENV: LED -> ON | cycle=5
(5770) LG_ENV: LED -> OFF | cycle=5

```

Figure 15: Monitor output with timestamps confirming the 1 Hz period.

7 Expected Results

Expected Results

- **SCADE Simulation:** Not applicable — no SCADE model exists in this practice. The equivalent verification artifact is a clean `idf.py` build with zero warnings and a successful `idf.py set-target esp32` configuration.
- **Console / UART Output:** The monitor must display the boot banner once, then the transition log with monotonically increasing timestamps and cycle count:


```

I (312) LG_ENV: lg_env_check | ESP-IDF v5.5.4
I (322) LG_ENV: LED -> ON | cycle=0
I (822) LG_ENV: LED -> OFF | cycle=0
I (1322) LG_ENV: LED -> ON | cycle=1
I (1822) LG_ENV: LED -> OFF | cycle=1

```
- **Physical Behavior:** The on-board LED blinks steadily at 1 Hz (500 ms ON / 500 ms OFF), synchronized with the LED -> ON / LED -> OFF messages in the monitor. Pressing the EN button restarts the firmware: the banner reappears and the cycle counter resets to 0.

8 Assessment Questionnaire

Answer each question based on your specific implementation. Justify your responses with references to your design decisions.

Q1. [Comprehension] In your `lg_env_check` project, trace what happens between typing `idf.py` build and the appearance of `lg_env_check.bin` in the `build/` folder. Name

each tool in the pipeline and the artifact it consumes and produces.

Q2. [*Analysis*] In your `main.c`, predict what would happen to both the LED and the monitor output if the *second* `vTaskDelay(pdMS_TO_TICKS(500))` (the one after `LED -> OFF`) were deleted. Would the LED still appear to blink? What period would the timestamps show? Justify using the loop structure, then verify on your board.

Q3. [*Analysis*] The transition log could have been produced with `printf` instead of `ESP_LOGI`. Compare both alternatives for this project specifically — consider timestamps, per-tag filtering, and what happens to each in Practice 3 when SCADE-generated code shares the same UART. Argue for one.

Q4. [*Synthesis*] A new requirement states that the LED must blink at 1 Hz but the `LED -> ON/OFF` messages must be reported only once every 10 cycles, keeping the cycle counter accurate. Modify the loop in your `main.c` to satisfy it using only the constructs already present in this practice, and write out the resulting monitor output for the first 20 cycles.

Q5. [*Critical*] Your project reports its state only through UART0, which is also the channel used for flashing and shared by every ESP-IDF system message. Identify two ways this single shared channel could mislead an operator diagnosing the landing gear control

system in later practices, and propose one mitigation using only the `ESP_LOG` features described in Section 4.

9 Conclusion

Write a concise conclusion reflecting on what you implemented, what you observed, and what challenges you encountered. Connect your findings to the broader context of model-based design for critical systems — in particular, why a verified toolchain is a prerequisite for trusting any code generated from a SCADE model.

10 Bonus Challenge

★ Bonus Challenge

Challenge:

Extend `lg_env_check` so the blink pattern encodes a heartbeat diagnostic: instead of a plain 1 Hz blink, produce a double-pulse pattern (100 ms ON, 100 ms OFF, 100 ms ON, 700 ms OFF — a 1 Hz period overall) and log the pattern phase (`PULSE_1`, `GAP`, `PULSE_2`, `IDLE`) with the same `LG_ENV` tag. Use only the APIs from this practice: `gpio_set_level`, `vTaskDelay`, and `ESP_LOGI`. Verify with the monitor timestamps that the full period still totals 1000 ms. **Hint:**

You do not need new control structures — think of the loop body as a fixed sequence of (level, duration, label) segments and consider how an array of such segments keeps the loop body to a few lines.

11 Troubleshooting

This section is populated with real issues encountered during development. If you run into a problem not listed here, document it using the same format below.

Issue: First `idf.py` command appears to hang

Symptom: The very first `idf.py -version` (or any first `idf.py` invocation after installing) sits with no output for a minute or longer.

Cause: On first run the Python virtual environment is cold and Windows Defender scans every freshly installed executable in `C:\Espressif`.

Fix: Wait up to two minutes. If it is still stuck, press `Ctrl+C`, close the terminal, reopen “ESP-IDF 5.5 CMD” and retry — the second run completes in seconds because everything is cached.

Issue: Commands rejected: “not recognized as a command”

Symptom: `xcopy` or `mkdir` fails, or fragments of the path are interpreted as separate commands.

Cause: Spaces were typed around the backslashes or inside `%IDF_PATH%` (e.g. `% IDF_PATH %`). The Windows shell splits arguments on spaces, so each fragment becomes a separate token.

Fix: Type each path as one unbroken string, exactly as printed in this manual: `%IDF_PATH%\examples\get-started\hello_world`. Copy-paste the command instead of retyping it.

Issue: Project folders accidentally created inside C:\Espressif

Symptom: `mkdir` was run from the default prompt (`C:\Espressif\frameworks\esp-idf-v5.5.4>`) and stray project folders appeared inside the toolchain tree. `mkdir` may also report that `components` already exists — that one is a legitimate ESP-IDF folder.

Cause: The ESP-IDF terminal always opens inside the framework directory, not in the user's project folder.

Fix: Always run `cd C:\esp` before creating or copying projects. Remove only the stray folders you created (e.g. `rmdir /s /q lg_env_check`); never delete `components`, `examples`, or any other pre-existing ESP-IDF folder.